ADR 0107 — An activity cursor names one fold, not an offset in a live feed

Status: Accepted — implemented; verification and mutation evidence below **Date:** 2026-09-02 **Issues:** #559, #562 **Follows:** ADR 0002 (wire-protocol negotiation), ADR 0013 and the M1.10 paged-history contract (signed, scoped cursors) **Supersedes:** the undocumented whole-feed `MAX_LIMIT = 500` behavior **Superseded by:** nothing

---

Context

`GET /api/activity` used to return a bare `Vec<ActivityEvent>`. Its only query field was `limit`, silently clamped to 500 before `assemble_feed` folded the app journal, every reflog and synthesized snapshot-diff events. Once the fold held 501 rows, row 501 was unreachable. A short answer could mean either “the journal ended” or “the server cut the journal off”; the response carried no fact that distinguished them.

The cap occurred twice. The query handler assembled with the clamped limit, and `GET /api/undoables/{id}` independently assembled with 500 while finding the hint for a tapped commit. Removing only the first cap would make an old event visible but could still hide the undo action derived from that same fold.

Activity is harder to page than one append-only table. The result is a derived sequence:

1. reflog lines are parsed and rebase bursts coalesced;
2. matching HEAD/branch movements collapse;
3. journal entries absorb corresponding reflog entries and supply attribution;
4. remote-ref update bursts fold into one fetch/pull row;
5. undo hints are recomputed against current refs and remote reachability;
6. events are stably sorted newest-first.

A new event at the head can therefore do more than shift indexes. It can change which older source records coalesce into a displayed event. Paging by an offset in “whatever the fold looks like now” would silently skip or repeat rows.

The response change is incompatible: a protocol-v9 browser expects an array and cannot deserialize an object. ADR 0002 therefore requires a hard protocol window move to v10 rather than a late Activity-panel failure after successful startup negotiation.

Decision

1. Fold completely, then page the result

Both server call sites pass `usize::MAX` to `assemble_feed`. The existing bounded source reads remain: the journal reader exposes its newest 1,000 lines and the reflog reader exposes 200 entries per ref. This ADR removes the additional 500-row *folded-result* ceiling; it does not claim an unbounded disk read.

`limit` now means page size. It defaults to 100 and is clamped into `1..=500`. Zero is promoted to one so a caller cannot create a non-advancing page loop.

`undoables` is not paginated because it answers one commit-specific question. It searches the complete available fold before selecting matching hints. Thus an activity row made reachable after 500 retains the same undo behavior it had when it was near the head.

2. A cursor is a signed position in one exact fold

For each request the server serializes the complete, final `Vec<ActivityEvent>` and computes:

```
activity-v1:sha256(exact serialized folded feed)
```

The existing per-process `CursorCodec` signs an envelope containing:

- the opaque repository/worktree scope;
- that activity generation token; and
- the absolute index of the next event in that exact fold.

The index is safe only because it is inseparable from the fold digest. On a cursor request the server authenticates the cursor, checks its repository scope, recomputes the complete fold and compares the generation before slicing. Any change — a new head event, removed event, different coalescing, changed undo hint or changed captured refs — returns HTTP 409 with a direction to restart at page one. A forged, malformed, cross-repository or out-of-range cursor returns HTTP 400. A process restart rotates the signing key and invalidates old cursors, as paged history already does.

This deliberately chooses consistency over a best-effort live scroll. A caller may need to restart if a busy repository changes while it walks, but it will never receive a plausible-looking aggregate that omitted or duplicated an event.

3. Exhaustion is explicit in a shared page envelope

`git-vista-protocol` owns the generic response:

```
pub struct ActivityPage<E> {
    pub events: Vec<E>,
    pub cursor: Option<String>,
}
```

`Some(cursor)` means another page exists. `None` means this page reached the end. The field is serialized as `null`, not omitted, so equal-length pages at the cutoff and at the end remain visibly different wire states.

The MCP tool returns one page per call, advertises `cursor`, and passes it to the endpoint unchanged. Its `limit` description now says “per page” rather than “capped feed.” The browser follows cursors to `None` each time the Activity panel opens, preserving the panel’s existing whole-list rendering while removing its identical 500-row ceiling. A repeated cursor is refused locally instead of spinning forever.

4. Protocol v10, no route change

The bare array becomes an object, so `PROTOCOL_VERSION`, `MIN_CLIENT_PROTOCOL`, and `MAX_CLIENT_PROTOCOL` move together from 9 to 10. No URL or method changes. `/api/activity` retains its existing authorization classification and route count; `route_authz.rs` is untouched.

Alternatives considered

Raw folded offset

Rejected. Inserting one event at the head shifts every later offset. More importantly, a new source record can change the fold itself. Applying offset 500 to the new fold would skip or repeat without any detectable error.

Timestamp, object id, or displayed-event key

Rejected. Reflogs have one-second timestamps, and multiple genuinely distinct events can have the same timestamp, kind, ref and object ids. Displayed rows do not carry a durable unique id, and coalescing can replace several old source records with one new row. Adding an occurrence ordinal recreates the unstable offset problem among identical keys.

Server-held snapshot identified by a random token

Rejected. It would make correctness depend on cache eviction, session cleanup, memory budgets and affinity between listeners. The signed fold digest is stateless, uses the existing shared codec, and makes restart invalidation an explicit behavior instead of leaked lifecycle state.

Encode the complete feed in the cursor

Rejected. It avoids a re-fold but makes query strings grow with journal size, exposes feed data into URLs/logs and defeats the cursor decoder’s fixed input bound.

Continue returning an array and add headers

Rejected. The MCP result would lose the continuation fact when it parses and reserializes the body, and a caller should not need transport-specific header access to learn whether a data sequence ended. The fact belongs beside the events in the shared DTO.

Consequences

Good: every folded event in the server’s bounded source windows is reachable; truncation is never silent; live drift is refused rather than spliced; browser, MCP and direct HTTP callers share one contract; old clients are rejected during negotiation; undo hints obey the same reachability rule.

Costs: each page re-reads and re-folds the available source windows, hashes the complete result and then returns a slice. A full walk is therefore quadratic in serialized/folded work. The present hard bounds (1,000 journal lines and 200 reflog lines per ref) make that cost finite, and this favors a stateless correctness contract over a cache with new ownership and eviction failure modes. A changing repository can force a caller to restart.

Decision log and acceptance evidence

The acceptance fixture writes 620 independently named journal events, above the former ceiling, and reads the real repository sources through the production collector on every page. It requests 137 rows per page and terminates after five pages. Its expected list is built independently as `paging-619` through `paging-000`; it does not ask the fold under test what “complete” means. The assertion that would fail if the fold dropped entries is the exact equality between that 620-name expected list and the collected list. A uniqueness set also has to contain 620 names, so duplicated rows cannot compensate for a missing one.

Named invariants and their direct tests:

Invariant	Evidence
Every available folded event is returned exactly once, in stable newest-first order, and the walk terminates	`more_than_500_folded_events_are_walked_once_in_order_and_terminate`
A cursor never applies its offset to a changed live fold	`a_cursor_refuses_a_feed_that_changed_at_the_head`
A cursor is authenticated and bound to one repository/worktree scope	`an_activity_cursor_is_authenticated_and_bound_to_its_repository`
The wire tells “more” from “end”	`the_wire_distinguishes_more_events_from_the_end` plus the first/final cursor assertions in the 620-event walk
The second, undo-hints fold also searches past 500	`undo_hints_search_past_the_former_500_event_ceiling`
MCP actually forwards both paging fields, rather than merely advertising them	`activity_path_passes_the_opaque_cursor_and_page_limit`
A cached incompatible client is stopped at negotiation	`protocol_v10_is_a_hard_compatibility_window`

Mutation matrix

`failure-atlas mutation_check` ran from commit `94df313f`. Every invariant was broken two different ways: one removes its mechanism, one weakens or misroutes it. Every unmutated baseline was green and every mutated leg reached and failed an assertion; no compiler failure is counted as a catch. The first invocation also demonstrated the tool's containment gate by refusing `/tmp`, which is a Git work tree on this host. Those

inconclusive refusal records are deliberately excluded from the evidence below; the conclusive rerun used the atlas's validated `/var/tmp/failure-atlas-codex` workspace.

Invariant	Remove mutation	Weaken/misroute mutation	failure-atlas result
complete, ordered, terminating walk	restore `MAX_PAGE_LIMIT` on the query fold	advance the encoded next position by one	caught/caught — records 141–142; exact 620-row equality fails
live-fold drift refusal	remove the generation comparison	hash only the unchanged tail event	caught/caught — records 143–144; the expected 409 becomes an `0k` page
authenticated, repository-bound cursor	remove the activity scope comparison	remove the shared codec's HMAC comparison	caught/caught — records 153–154; foreign and edited cursors respectively become `0k` pages
explicit more/end wire state	omit `cursor` when it is `None`	rename the wire field to `next_cursor`	caught/caught — records 145–146; exact JSON differs
undo hints past 500	restore `MAX_PAGE_LIMIT` on the undo fold	cap that fold at one	caught/caught — records 147–148; expected hint count is one, actual is zero
MCP cursor + limit forwarding	discard the cursor argument	serialize the cursor under `limit=`	caught/caught — records 149–150; exact request path differs
protocol v10 negotiation gate	move `PROTOCOL_VERSION` back to 9	weaken `MIN_CLIENT_PROTOCOL` to 9	caught/caught — records 151–152; exact whole-window assertions differ

Total: **14/14 conclusive catches**, two independent mutations for each named invariant.

Verification

- `cargo test -p git-vista-server -j 1`: **1,137 passed, 0 failed, 6 ignored** across the main binary, sandbox binary and four integration targets.
- `cargo test -p git-vista-protocol -j 8`: **226 passed, 0 failed, 1 ignored**.
- `cargo test -p git-vista-mcp -j 8`: **72 passed, 0 failed, 7 ignored**.
- `cargo check -p git-vista --bin git-vista-ui -j 8`: passed.
- `cargo check -p git-vista --bins --target wasm32-unknown-unknown -j 8`: passed, compiling the browser consumer on its real target.
- `cargo clippy --workspace -j 8 -- -D warnings`: passed.
- `cargo fmt --all -- --check` and `git diff --check`: passed.
- failure-atlas: **14 caught, 0 survived, 0 baseline/build failures** in the conclusive run (records 141–154).

Signed: max · 2026-09-02